

// devlog · entry 01

My Spring Boot Journey

from "how does a browser even talk to a server" → Spring Boot

STARTED

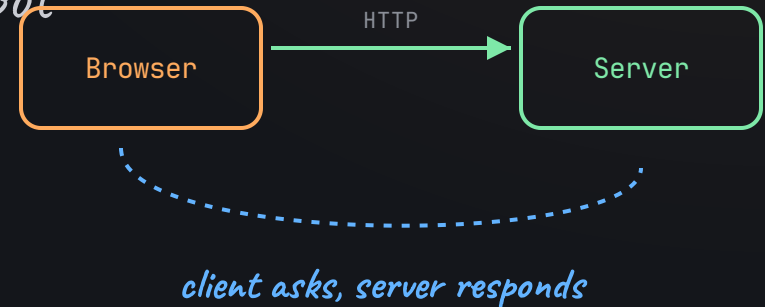
August 2026

TRACK

Java Backend

STATUS

Week 1 – Foundations



before any framework...

Two machines, one conversation

Everything — Spring, Spring Boot, Servlets — exists to answer one question: how does code on **my** machine talk to code on **another** machine?

Browser

Mobile App

Postman

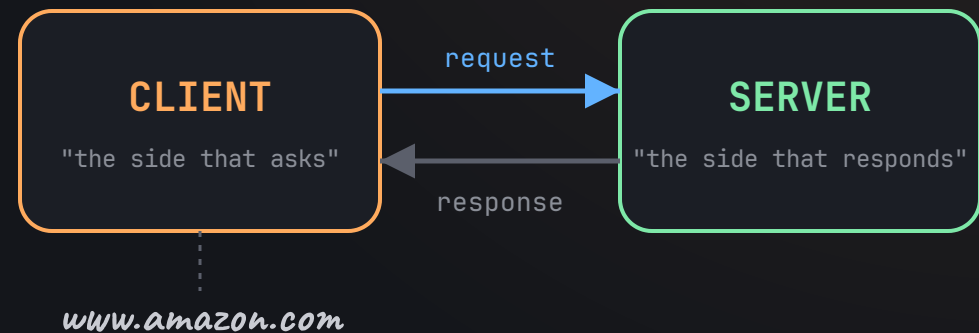
React FE

Amazon Server

Bank Server

Spring Boot App

"Client asks. Server responds." — the whole internet, basically.



so they don't talk gibberish

HTTP is the shared language

HyperText Transfer Protocol. A rulebook so client and server agree on how a request looks, how a response looks, and what status it ended with.

GET

POST

PUT

PATCH

DELETE

GET requests fetch data → no body needed.

POST / PUT / PATCH send data → body carries it, usually as **JSON**.

REQUEST

```
POST /login
Content-Type: json

{ email, password }

method · url · headers · body
```

RESPONSE

```
200 OK
Content-Type: json

{ "message": "Login ok" }

status · headers · body
```

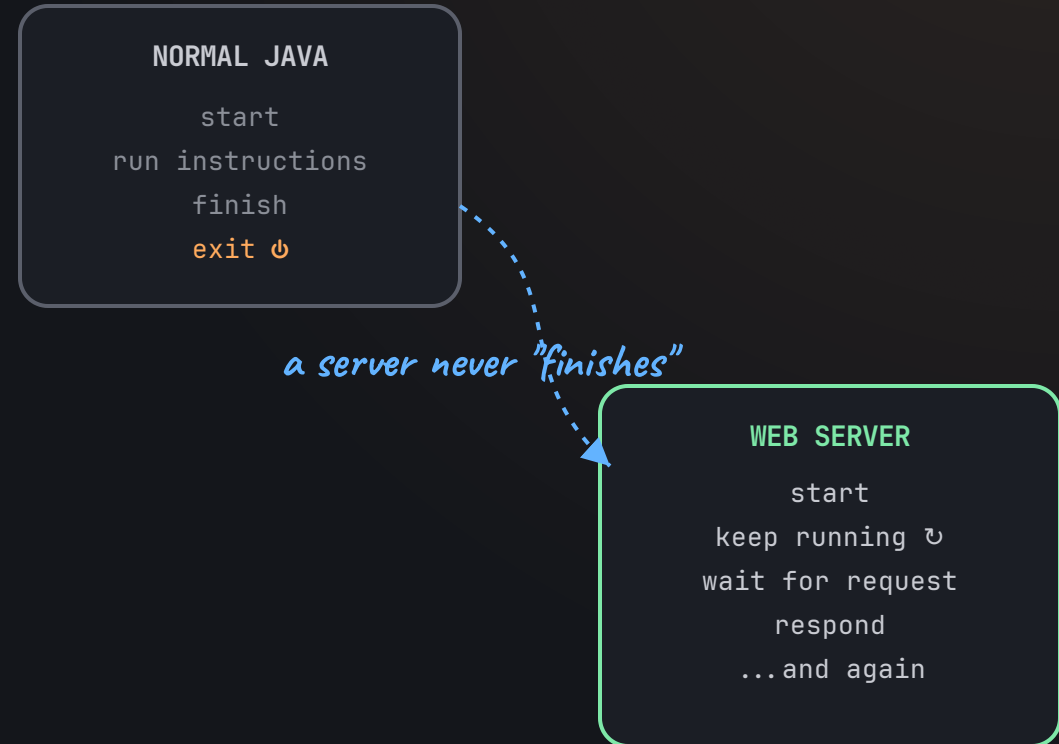
normal java program

Run once. Exit. That's not a website.

A .java file compiles to bytecode, the JVM runs it, it prints something, it exits. **A server can't do that** — it has to stay alive and keep listening.

Normal program — start → run → finish → exit

Web server — start → keep running ∞
wait for requests → respond → repeat forever



core java can open a socket, but...

To the JVM, HTTP is just bytes

`new ServerSocket(8080)` works — Java can listen on a port. But a raw `GET /users HTTP/1.1` means nothing to it until **someone parses it by hand**.

building this ourselves = repeating the same plumbing on every project.

If we did it in raw Core Java, we'd manually...

- 01 open a port • `ServerSocket`
- 02 read raw input streams
- 03 parse the HTTP request text
- 04 pull out method / url / headers / body
- 05 route to the right Java method
- 06 build the HTTP response by hand
- 07 manage threads for many users
- 08 handle malformed requests, errors...

none of this is actual business logic.

→ that's why a standard solution was needed.

something had to sit in between

Servlets translate HTTP → Java

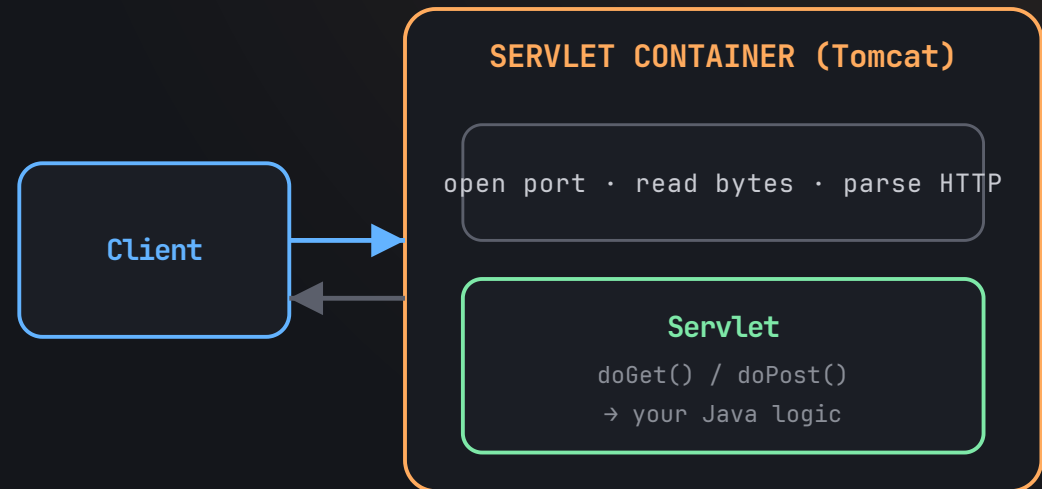
A **Servlet** is a Java class built to handle HTTP requests. It runs inside a **Servlet Container**, which does all the low-level web plumbing for us.

Apache Tomcat

Jetty

Undertow

The container: opens the port • reads bytes • parses HTTP • creates request/response objects • manages threads • calls doGet() / doPost()



servlets solved http... but

Big apps built on raw Servlets got messy

Too much boilerplate. Too much configuration. Classes tightly glued together. Hard to test, hard to maintain, the same plumbing rewritten on every project.

Developers needed a cleaner way to organize code, manage objects, and wire dependencies — that's the gap Spring Framework stepped into.

~~too much boilerplate~~

~~too many configs~~

~~tight coupling~~

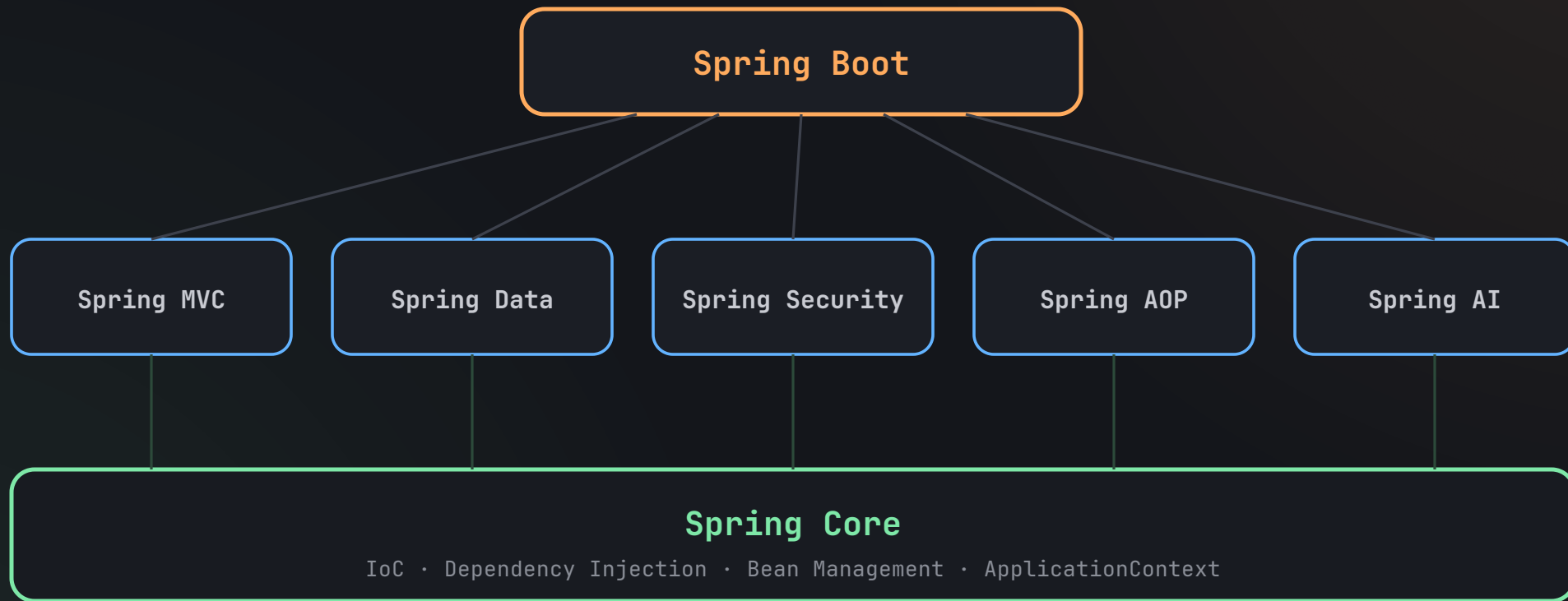
~~difficult testing~~

~~repeated code, every project~~

enter Spring →

not a library. an ecosystem.

Spring Boot sits on top of Spring Core

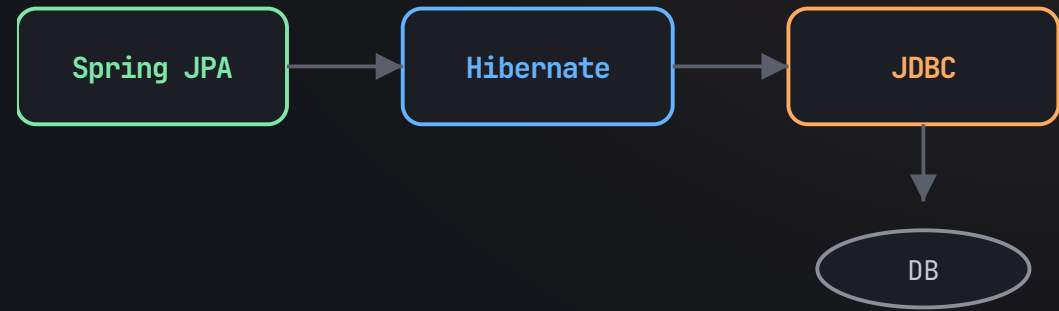


"Spring Boot uses Spring. It doesn't replace it."

jpa is a spec. hibernate does the work.

JDBC crawled, Hibernate walked, Spring Data JPA flies

JDBC meant writing SQL and managing connections by hand. Hibernate mapped Java objects to tables (ORM). Spring Data JPA wraps it all and removes even more boilerplate.



the complete journey

From a browser click to my business logic



HTTP request comes in on the left · a clean method call is all I write on the right

everything left of "My Code" is what Spring Boot just... handles.

so far, so good

Spring Boot doesn't feel magic anymore

- ✓ Client asks, server responds
- ✓ HTTP is just the agreed format
- ✓ Core Java alone can't parse HTTP
- ✓ Servlets translate HTTP → Java calls
- ✓ Spring Core manages objects (IoC/DI)
- ✓ Spring MVC = clean web layer
- ✓ Spring Data JPA → Hibernate → JDBC
- ✓ Spring Boot = auto-config on top of it all

next up: writing my first @RestController and actually shipping something.